

```
for (key <- j.select("COD")) { println(key) }
```

Para demais funções disponíveis digite **j.** e pressione TAB duas vezes.

Transformações

A tabela a seguir lista algumas das transformações mais comuns implementadas pelo Spark. Para mais detalhes recorra à API dos RDDs (Scala (<http://spark.apache.org/docs/latest/api/scala/index.html#org.apache.spark.rdd.RDD>), Java (<http://spark.apache.org/docs/latest/api/java/index.html?org/apache/spark/api/java/JavaRDD.html>), Python (<http://spark.apache.org/docs/latest/api/python/pyspark.html#pyspark.RDD>), R (<http://spark.apache.org/docs/latest/api/R/index.html>)) e para a documentação das funções para pares RDD (Scala (<http://spark.apache.org/docs/latest/api/scala/index.html#org.apache.spark.rdd.PairRDDFunctions>), Java (<http://spark.apache.org/docs/latest/api/java/index.html?org/apache/spark/api/java/JavaPairRDD.html>)).

Transformação	Descrição	Exemplo	Resultado
map(func)	retorna um RDD depois de processar cada elemento com a função <i>func</i>	sc.parallelize(Array(1, 2, 3, 4, 5)).map (x => x + 10)	{ 11, 12, 13, 14, 15 }
filter(func)	Retorna um RDD depois de selecionar os enementos onde <i>func</i> é verdadeira	sc.parallelize(Array(1, 2, 3, 4, 5)).filter(x => x == 3)	3
flatMap(func)	Similar ao map, mas cada item pode ser mapeado para 0 ou mais itens (<i>func</i> deve retornar uma sequência, ao invés de um item).	sc.parallelize(List("this is line 1", "this is the second line"), 2).flatMap(_._split(" ")).foreach {println _}	{this, is, the, second, line, this, is, line, 1}
mapPartitions(func)	Similar ao map, mas executa <i>func</i> separadamente em cada partição (bloco). A função <i>func</i> deve ser do tipo <i>Iterator<T> => Iterator<U></i> quando invocada sobre um RDD do tipo T.	sc.parallelize(Array((1,2), (1,2), (1,2), (3,4), (5,6), (7,8), (7,8)), 3).mapPartitions (x => x.map((t: (Int, Int)) => t._1 + t._2)).foreach {println }	{3, 3, 3, 7, 11, 15, 15}
mapPartitionsWithIndex(func)	Similar a mapPartitions, mas provê <i>func</i> com um valor inteiro representando o índice da partição. A função <i>func</i> deve ser do tipo <i>Iterator<T> => Iterator<U></i> quando invocada sobre um RDD do tipo T.	sc.parallelize(Array((1,2), (1,2), (1,2), (3,4), (5,6), (7,8), (7,8)), 3).mapPartitionsWithIndex ((index: Int , x) => x.map((t: (Int, Int)) => (index, t._1 + t._2))).foreach {println }	(0,3) (0,3) (1,3) (1,7) (2,11) (2,15) (2,15)
sample(withReplacement, fraction, seed)	Amostragem de uma fração de um dado, com ou sem substituição, usando um número aleatório.	sc.parallelize(Array(1, 2, 3, 4, 5)).sample(false, 0.4).foreach { println _ }	{3,4,2}
union(otherDataset)	Retorna uma coleção que contém a união de todos os elementos da fonte e do argumento.	sc.parallelize(Array(1, 2, 3, 4, 5)).union(sc.parallelize(Array(6,7,8,9,10))).foreach(println _)	{1,2,3,4,5,6,7,8,9,10}
intersection(other Dataset)	Retorna um RDD que contém uma interseção dos elementos da fonte e do argumento.	sc.parallelize(Array(1, 2, 3, 4, 5)).intersection(sc.parallelize(Array(4,5,6,7,8))).foreach(println _)	{5,4}
distinct([numTasks])	Retorna uma coleção que contém os elementos distintos da fonte.	sc.parallelize(Array((1,2), (1,2), (1,2), (3,4), (5,6), (7,8), (7,8))).distinct().foreach { println _ }	{{(1,2), (5,6), (7,8), (3,4)}
groupByKey([numTasks])	Quando invocado sobre uma coleção de pares (k, v), retorna um par (k, Iterable<v>).	sc.parallelize(Array((1,2), (1,2), (1,2), (3,4), (5,6), (7,8), (7,8))).groupByKey().foreach { println _ }	(1,CompactBuffer(2, 2, 2)) (3,CompactBuffer(4)) (7,CompactBuffer(8, 8)) (5,CompactBuffer(6))
reduceByKey(func, [numTasks])	Quando invocado sobre uma coleção de pares (k, v), retorna uma coleção de pares (k, v) onde os valores de cada chave são agregados usando pela função <i>func</i> , que deve ser do tipo (v, v).	sc.parallelize(Array((1,2), (1,2), (1,2), (3,4), (5,6), (7,8), (7,8))).reduceByKey(_ + _).foreach { println _ }	(1,6) (3,4) (7,16) (5,6)

aggregateByKey (<i>zeroValue</i>)(<i>seqOp</i> , <i>combOp</i> , [<i>numTasks</i>])	Quando invocado sobre uma coleção de pares (k, v) a função <i>seqOp</i> é aplicada nos valores v de cada chave. Depois, função <i>combOp</i> é invocada para agregar os valores resultantes de <i>seqOp</i> . O <i>zeroValue</i> representa o valor zero (ponto inicial) para calcular o valor de retorno da função <i>combOp</i> . Esta função permite que o tipo do valor de retorno seja diferente do tipo valor de entrada. O tipo do valor de retorno é definido por <i>zeroValue</i> .	sc.parallelize(Array((1,2), (1,2), (1,2), (3,4), (5,6), (7,8), (7,8)), 3).aggregateByKey(List[Int]())((x,y) => x ++ List(y) , _ ++ _).foreach {println}	(1,List(2, 2, 2)) (3,List(4)) (7,List(8, 8)) (5,List(6))
sortByKey ([<i>ascending</i>], [<i>numTasks</i>])	Quando invocado sobre uma coleção de pares (k, v), onde k implementa Ordered, retorna uma coleção de pares (k, v) ordenados pela chaves.	sc.parallelize(Array((1,2), (1,2), (1,2), (3,4), (5,6), (7,8), (7,8)), 1).sortByKey().foreach {println _ }	(1,2) (1,2) (1,2) (3,4) (5,6) (7,8) (7,8)
join (<i>otherDataset</i> , [<i>numTasks</i>])	Quando invocado sobre uma coleção de pares do tipo (k, v) e (k, w), retorna uma coleção de pares (k, (v, w)).	sc.parallelize(Array ((1, "um"), (2, "dois"), (3, "tres"))) .join(sc.parallelize(Array ((1, "one"), (2, "two"), (3, "three")))).foreach { println _ }	(1,(um,one)) (3,(tres,three)) (2,(dois,two))
cogroup (<i>otherDataSet</i> , [<i>numTasks</i>])	Mesmo que <i>groupWith</i> Quando invocado sobre uma coleção de pares (k, v) e (k, w), retorna uma coleção do tipo (k, (Iterable<v>,Iterable<w>)).	sc.parallelize(Array ((1, "um"), (2, "dois"), (3, "tres"))) .cogroup(sc.parallelize(Array ((1, "one"), (2, "two"), (3, "three")))).foreach { println _ }	(2, (CompactBuffer(dois),CompactBuffer(two))) (1, (CompactBuffer(um),CompactBuffer(one))) (3, (CompactBuffer(tres),CompactBuffer(three)))
cartesian (<i>otherDataset</i>)	Quando invocado sobre uma coleção de dados do tipo T e U, retorna o conjunto de pares (T, U).	sc.parallelize (1 to 2).cartesian(sc.parallelize(10 to 11)).foreach {println _ }	(1,10) (1,11) (2,10) (2,11)
pipe (<i>command</i> , [<i>envVars</i>])	Concatena cada partição do RDD por meio de um comando shell e.g. um script Perl ou Bash. Elementos do RDD são escritos para a stdin do processo.	ramiro\$ cat script.sh #!/bin/bash while read line; do echo "(script, \$line)"; done sc.parallelize (0 to 5).pipe ("/Users/Edson/Development/spark-training/usb/spark-training/spark/script.sh").foreach {println _ }	(script, 3) (script, 4) (script, 5) (script, 0) (script, 1) (script, 2)
coalesce (<i>numPartitions</i>)	Diminui o número de partições de um RDD para <i>numPartitions</i> .	scala> val d = sc.parallelize (1 to 100, 4) scala> d.partitions.size res29: Int = 4 scala> d.coalesce (2).partitions.size res32: Int = 2	4, 2
repartition (<i>numPartitions</i>)	Distribui novamente os dados de um RDD aleatoriamente para criar, ou mais, ou menos partições.	scala> val d = sc.parallelize (1 to 100, 4) scala> d.partitions.size res29: Int = 4 scala> scala> d.repartition(2).partitions.size res36: Int = 2	4, 2

. Ações

A tabela a seguir lista algumas das ações mais comuns implementadas pelo Spark. Para mais detalhes recorra à API dos RDDs (Scala (<http://spark.apache.org/docs/latest/api/scala/index.html#org.apache.spark.rdd.RDD>), Java (<http://spark.apache.org/docs/latest/api/java/index.html#org/apache/spark/api/java/JavaRDD.html>), Python (<http://spark.apache.org/docs/latest/api/python/pyspark.html#pyspark.RDD>), R (<http://spark.apache.org/docs/latest/api/R/index.html>)) e para a documentação das funções para pares RDD. (Scala (<http://spark.apache.org/docs/latest/api/scala/index.html#org.apache.spark.rdd.PairRDDFunctions>), Java (<http://spark.apache.org/docs/latest/api/java/index.html#org/apache/spark/api/java/JavaPairRDD.html>)) for details.

Ações	Descrição	Exemplo	Resultado
reduce(func)	Agrega os elementos de uma coleção usando a função <i>func</i> (que precisa de dois argumentos, retornando um). A função deve ser acumulativa ou associativa, para ser computada corretamente em paralelo.	sc.parallelize(Array(1, 2, 3, 4, 5)).reduce((a, b) => a+b)	15
collect()	Retorna todos os elementos de uma coleção como um vetor dentro do programa <i>driver</i> .	sc.parallelize(Array(1, 2, 3, 4, 5)).collect	Array(1, 2, 3, 4, 5)
count()	Retorna o número de elementos dentro da coleção de dados.	sc.parallelize(Array(1, 2, 3, 4, 5)).count	5
first()	Retorna o primeiro elemento da coleção de dados.	sc.parallelize(Array(1, 2, 3, 4, 5)).first	1
take(n)	Return an array with the first <i>n</i> elements of the dataset.	sc.parallelize(Array(1, 2, 3, 4, 5)).take(2)	Array(1, 2)
takeSample(withReplacement, num, [seed])	Retorna um vetor com uma amostra aleatória de <i>num</i> selecionando opcionalmente uma semente para os números aleatórios. O parâmetro <i>withReplacement</i> diz respeito a repetir os elementos na amostra ou não.	sc.parallelize(Array(1, 2)).takeSample(true,4)	Array(1, 1, 2, 1)
takeOrdered(n, [ordering])	Retorna os primeiros <i>n</i> elementos de um RDD usando ou a ordem natural ou um comparador customizado.	sc.parallelize(Array(1, 2, 3, 4, 5)).takeOrdered(3)	Array(1, 2, 3)
saveAsTextFile(path)	Escreve todos os elementos da coleção de dados em um arquivo texto no sistema de arquivos local ou em qualquer sistema de arquivos suportado pelo Hadoop.	sc.textFile("README.md").saveAsTextFile("README.md2")	README.md2/_SUCCESS README.md2/part-00000 README.md2/part-00001
saveAsSequenceFile(path) (Java and Scala)	Escreve todos os elementos da coleção de dados como um <i>SequenceFile</i> do Hadoop no sistema de arquivos local, ou em qualquer sistema de arquivos suportado pelo Hadoop.	sc.parallelize(Array((1,2), (1,2), (1,2), (3,4), (5,6), (7,8), (7,8))).saveAsSequenceFile("SEQ")	SEQ/_SUCCESS SEQ/part-00000 SEQ/part-00001
saveAsObjectFile(path) (Java and Scala)	Escreve todos os elementos da coleção de dados em um formato simples usando serialização Java, que pode ser deserializada posteriormente usando <i>SparkContext.objectFile()</i> .	sc.textFile("README.md").saveAsObjectFile("README.obj")	README.obj/_SUCCESS README.obj/part-00000 README.obj/part-00001
countByKey()	Somente disponível em RDDs do tipo (k, v) hashmap de pares (K, Int) com o número de ocorrência de cada chave.	sc.parallelize(Array((1,2), (1,2), (1,2), (3,4), (5,6), (7,8), (7,8))).countByKey	Map(1 -> 3, 3 -> 1, 7 -> 2, 5 -> 1)
foreach(func)	Executa uma função <i>func</i> em cada elemento da coleção de dados. Geralmente utilizada para atualizar contadores ou interagir com sistemas de arquivos externos. Observação: modificar <i>accumulators</i> fora de um <i>foreach</i> pode um comportamento indefinido.	sc.parallelize(Array(1, 2, 3, 4, 5)).foreach { println _ }	{1,2,3,4,5}